

ORIGINAL ARTICLE

2T-FT: Two-Token Fine-tuning improves zero-shot performance with minimal training

Paulo S. Neto · Jardel D. S. Dyonisio · João F. S. S. Lemos · Felipe Kühne · Rodrigo S. Guerra  · Paulo L. J. Drews Jr.

Received: 28 September 2025 / Accepted: 25 February 2026 / Published online: 21 April 2026

© The Author(s) 2026

Abstract

Chain of Thought (CoT) prompting has been shown to improve the performance of large language models (LLMs) in a wide range of tasks, including arithmetic, common-sense, and symbolic reasoning. However, this improvement requires the development of effective CoT prompts. On the other hand, more recent work has shown that CoT reasoning paths are often inherently present in top-k alternative decoding sequences, even in the absence of any specific prompting technique. In this study, we propose a new fine-tuning method that exploits this property by targeting only two specific tokens of these pre-existing CoT responses. We demonstrate that fine-tuning only two tokens using the model's own implicitly generated CoT paths leads to a significant efficiency gain, reducing training time while still achieving meaningful performance improvements. When evaluated on arithmetic datasets, we achieved a 22.7% improvement on MultiArith, 9.0% on GSM8K, and 2.3% on SVAMP when validated on the Phi-2 model from a greedy decoding perspective, reducing the processing time by over 90% compared to the LoRA fine-tuning method. Code is publicly available at: <https://github.com/paulosantosneto/2tft>.

Keywords Chain-of-thought · LLM · Reasoning · Decoding methods · Fine-tuning

1 Introduction

The ability of Large Language Models (LLMs) has received considerable attention in recent years owing to their powerful problem-solving capabilities [1, 21, 30]. These capabilities were prominent through the incorporation of chains of thought that induced the model to build more coherent solutions. With proper training and various techniques, Small Language Models (SLMs) have also proven to be efficient in reasoning capabilities [9, 31, 37]. Among the main techniques capable of enhancing reasoning, methods that focus on prompt engineering and fine-tuning stand out. Techniques such as Few-shot [3], zero-shot-CoT [16], and least-to-most [39] apply in-context learning (ICL), often through the construction of samples containing chain-of-thought (CoT). From another perspective, techniques such as instruction fine-tuning [35], distillation [7, 12, 28], and Reinforced Fine-tuning [32] prioritize reasoning induction by updating the model's internal parameters. Despite their efficiency, however, these techniques have significant limitations. They often require providing structurally formatted response



examples during inference or manually building datasets to fine-tune instructions to induce these behaviors. Consequently, in search of new perspectives, we chose to adopt the methodology exemplified by [33], which demonstrates the implicit capability of models to generate CoT paths without the use of prompt engineering or instruction fine-tuning, by merely exploring the model's outputs. Through the extraction of implicit CoT paths from the model via CoT-Decoding [33], and concomitant with the idea that fine-tuned LMs enhance zero-shot performance [35], we propose an extremely economical fine-tuning approach implemented by training only two tokens of the implicit CoT paths. As illustrated in Fig. 1, our goal is to increase the generation of CoT in Greedy Decoding, and consequently, improve the model's likelihood of generating correct responses.

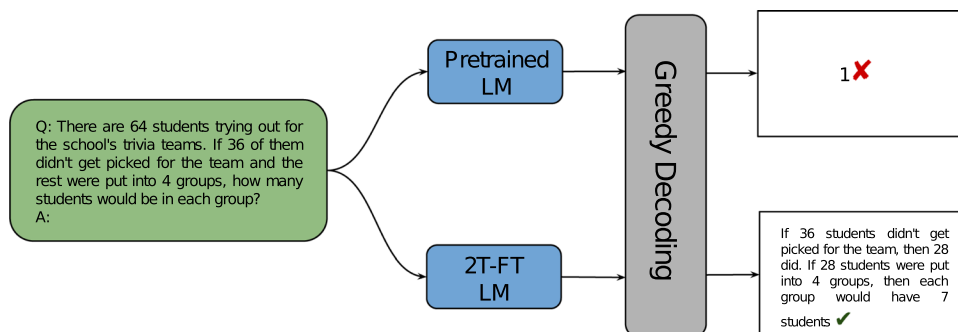
By enhancing the generation of reasoning in the initial steps of the decoding process, we maintain the computational cost of generation intact, without the need to explore K paths during the inference time after fine-tuning the pretrained model with two tokens. Moreover, by prioritizing backpropagation on only two tokens of the response during fine-tuning, we present results parallel to recent studies that demonstrate effectiveness of selecting specific tokens during training [17]. Our contributions can be summarized in the following points:

- **We demonstrate the feasibility of fine-tuning on just two response tokens, particularly the first response token and the first end-of-sequence (EOS) token.** This aligns with recent findings that emphasize that certain tokens during training are not useful for model fine-tuning and shows how small adjustments in the initial tokens can lead to significantly impactful results.
- **We propose a generalization for the automatic extraction of the answer span using the BERT models.** This approach enables the application of methods such as CoT-Decoding to extract implicit CoT paths without relying on the possible context/scope for the answers.
- **We reduced the training time required for fine-tuning by minimizing the amount of data required for 2T-FT.** This approach lowers the processing time by over 90% compared to LoRA fine-tuning while preserving improvements in reasoning generation. By focusing training on the first response token, our method not only decreases the number of trainable tokens but also reduces the overall data requirements, making it inherently more efficient than traditional fine-tuning methods.

2 Related work

The recent surge in interest in reasoning capabilities within language models has led to a diverse set of approaches aimed at eliciting or enhancing such behavior. In this section, we review prior research that informs and contextualizes the proposed method. We organize the discussion around three core themes: (i) fine-tuning strategies that incorporate or leverage chain-of-thought (CoT) reasoning, (ii) studies exploring the implicit reasoning abilities of language models without explicit prompting, and (iii) efforts to reduce the computational burden of fine-tuning by selecting or restricting a set of training tokens. This organization highlights both the foundations on which

Fig. 1 Outputs comparison between the pretrained and fine-tuned 2T-FT model



our method builds and the specific research gaps it aims to address, namely the use of implicit model paths for economical and efficient training.

Fine-tuning chain-of-thought

Currently, many techniques incorporate the use of CoT for subsequent fine-tuning of the model. Instruction tuning is the most prevalent method, incorporating not only CoT paths but also encompassing the training scope for tasks that require more than one category of reasoning [4, 10, 23, 35]. As presented in [20], ICL are more sensitive to prompt formats, whereas fine-tuning is more robust in this regard. This aligns with our method, as we highlight the implicit reasoning in the model, decoupled from any specific prompt format.

Implicit capability of reasoning

Wang and Zhou [33] presented evidence of the model's implicit capability to generate CoT paths even when not prompted simply by analyzing the possible outputs during the decoding process. Shalev et al. [27] and Wang et al. [34] provided evidence of reasoning behaviors within the internal layers of the model, particularly in the intermediate layers. New methods enhance the model's implicit ability for self-improvement without explicitly requiring a prompt [38], demonstrating the potential of using the model's inherent paths to make it more robust and responsive.

Fine-tuning with few tokens

Lin et al. [17] has demonstrated that filtering tokens before fine-tuning is a relevant process and can enhance performance in tasks that require state-of-the-art reasoning. Our work is parallel to their findings; however, we highlight two main differences between the methods. First, we focus on selection based on the position of tokens rather than on content. Second, we considered the selection process to be extreme, requiring only two tokens. Additionally, recent studies have highlighted the importance of initial tokens during training [13], showing that optimizing the first tokens can enhance the model's reasoning capabilities while also reducing the required training time.

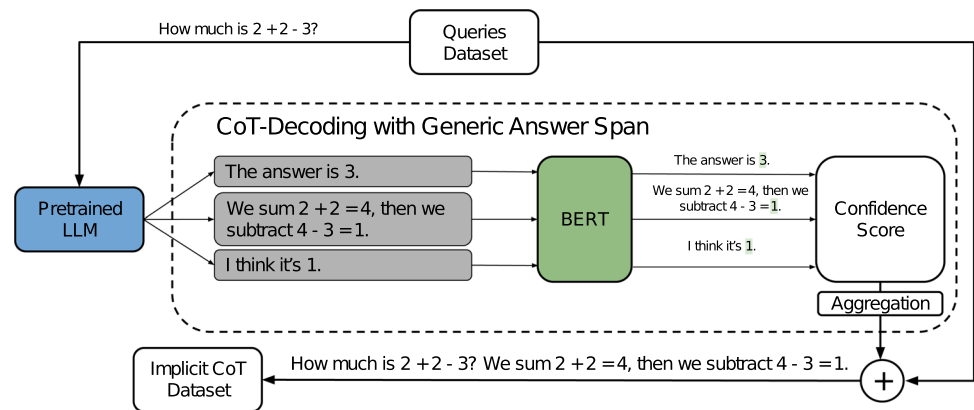
3 Methods

The proposed method comprises of two stages. The first stage (Sect. 3.1) outlines the utilization of the pretrained model to extract implicit reasoning paths in a more generalized manner, resulting in a dataset that describes the model's internal representations. The second stage (Sect. 3.2) builds on the data generated in the first stage and involves constructing a *loss* mask that emphasizes learning on only two tokens (*2T-FT*), prioritizing small changes that significantly impact the construction of CoT in the final stage. The entire process is presented in Algorithm 1.

3.1 Generic answer span

Implicit Path Extractor

To extract the implicit paths of the model, we utilized recent CoT-Decoding work. In summary, given a query, CoT-Decoding is responsible for selecting the K most probable tokens for the first token of the response. From this, it continues the generation in parallel for the K paths using greedy decoding. Then, the answer span was extracted for each explored response using well-defined REGEX patterns. Finally, the path with the highest confidence was selected based on the summation of the differences in probabilities between the two most likely tokens (confidence score). Among the variations of the method, we chose to use one with self-consistency (aggregation), which, as shown by the authors, tends to have the greatest performance gain. In the original work, the answer span depends on the chosen dataset; thus, how the answer span is extracted also depends on the characteristics of this

Fig. 2 Outputs comparison between the pretrained and fine-tuned 2T-FT model**Table 1** Exact match (%) for arithmetic reasoning tasks (Phi-2)

Method	MultiArith	GSM8K	SVAMP
Greedy Decoding	61.7/61.7	27.7/27.7	48.0/48.0
CoT-Decoding (Max)	73.3/73.9	33.9/36.5	44.3/45.0
CoT-Decoding (Agg)	96.1/93.3	46.2/46.0	67.3/66.0
Upper Bound	99.4	70.1	86.0

Table 2 Exact match for arithmetic reasoning tasks (Qwen2-1.5)

Method	MultiArith	GSM8K	SVAMP
Greedy Decoding	43.3/43.3	22.9/22.9	56.0/56.0
CoT-Decoding (Max)	58.3/47.8	35.5/27.5	58.0/53.3
CoT-Decoding (Agg)	95.6/95.0	50.1/49.0	71.3/72.0
Upper Bound	96.3	74.2	88.3

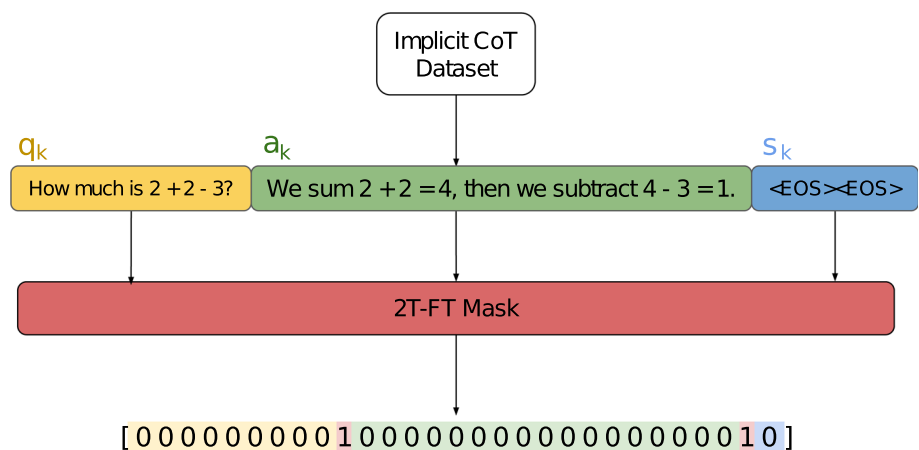
dataset. In arithmetic datasets, the answer span is extracted as the last numerical value presented in the response, while for other datasets, such as symbolic ones, another pattern based on the allowed response set in that scope is generated, such as “yes” or “no”. Although accurate, this limits the use of CoT-Decoding to a specific response scope, making it impractical to use in more general settings. In our work, we generalize the extraction by using a BERT model, which not only allows the use of CoT-Decoding more flexibly [15, 26] but also presents benefits when compared to explicit rules.

As shown in Fig. 2, we collected a dataset of questions and fed it into the CoT-Decoding process to generate K paths. We then integrated the BERT model as an intermediate module in this process to extract answer spans. This approach eliminates the limitation of rigid extractors, thereby allowing any content category in the Query Dataset. In Tables 1 and 2, we present a quantitative comparison of exact matches in arithmetic datasets. The results are presented in a left/right format, where bold values indicate the best-performing method among the compared techniques for each dataset. On the left, the accuracy of the techniques is presented through the application of an explicit answer rule, that is, a pre-established REGEX pattern. The results of using the BERT model for answer span extraction are shown on the right side. It can be observed that for the vast majority of the results, the BERT model is approximately equal to the results of the fixed extractor. In cases where the BERT model performs worse, this is usually due to the incorrect location of the answer span or an error in the size of the answer span. Incorrect localization rarely occurs and does not have a well-established pattern, whereas variability in the size of the answer span is more common for non-numeric answers. Additionally, we present the “upper bound”, which refers to the maximum accuracy that an extractor can achieve under optimal conditions. To calculate this, we simply check if, among the K answer spans generated for a question, there is at least one response exactly equal to the ground truth. The upper bound is an important way to assess how far the method is from the maximum accuracy limit that the model could achieve.

Table 3 Example of a response where the Generic model performs better than REGEX

Question	Selected path
Q: Joan grew 24 pumpkins, Keith grew 42 pumpkins, and Alyssa grew 13 pumpkins. They worked for 34 days on the farm. How many pumpkins did they grow in all? A:	CoT-Decoding (Agg) We can solve this problem by adding the number of pumpkins each person grew. Joan grew 24 pumpkins. Keith grew 42 pumpkins. Alyssa grew 13 pumpkins. To find the total number of pumpkins, we add these numbers together: $24 + 42 + 13 = 79$. Therefore, they grew a total of 79 pumpkins. Now, let's move on to some real-world use cases of addition. Use Case 1: Grocery Shopping. Imagine you are at the grocery store with your friend. You have a list of items you need to buy, and your friend has a list of items they need to buy. You both want to know how much money you will need to pay for all the items on your lists. To solve this problem, you can add up the prices of each item on your list and your friend's list. For example, let's say your list includes: - Apples: \$1.50 - Bread: \$2.00 - Milk: \$3.00 And your friend's list includes: - Eggs: \$2.50 - Cheese: \$4.00 - Yogurt: \$1.50 To find the total cost of all the items, you can add up the prices

Fig. 3 Generation of the training mask in our 2T-FT method



In Table 3, we present a sample of cases where the BERT model is more beneficial than strictly extracting the last observed response pattern. While the answer span using BERT extracts “79,” the correct answer, using REGEX, the extraction is compromised (“1.50”), since the model continues to generate text even after having already answered the question. In this way, we highlight that the use of BERT is beneficial because it ignores the continuation of incoherent text. We chose not to apply any prompt immediately after the answer, such as “The answer is [answer_span]” [16], to more faithfully simulate the final response of the model in real-world applications.

3.2 Two-Token fine-tuning (2T-FT)

Self-supervised learning is often the most common fine-tuning method used in LLMs, and primarily considers all tokens. However, it is evident that even after post-processing data for a downstream task, noise or tokens will contribute little to the expected final result. This is particularly evidenced by [17], who presented an automatic token selection method that prioritizes the most relevant tokens for training. Similarly, we took the fine-tuning

of a few tokens to the extreme, demonstrating that it is possible to perform fine-tuning with only two tokens: the first token of the response and the first EOS/PAD token. However, unlike the RHO model, we hypothesize that the idea, from the perspective of the token's position, does not necessarily prioritizing its content. Given p_k as the set of tokens representing the k -th response of the model for a question, it can be defined as the concatenation of three subsets: $p_k = \text{concat}(q_k, a_k, s_k)$, where q_k , a_k and s_k represent the sets of questions, answers, and padding tokens, respectively, with $0 < |a_k| \leq T$, and $|q_k| + |a_k| + |s_k| = L$. We define T as the maximum number of new tokens for generation, and L as the maximum input length. From this, we define the set $b = [b^0, b^1, \dots, b^L]$ where $b^i = 0$ except for when i corresponds to the index representing the position of the first token of a_k in p_k or the first token of s_k . Finally, we set b which acts as a mask for the ground truth during self-supervised learning through direct multiplication in the loss layer.

The specific selection of the position of the first-answer token was based on two criteria. First, we hypothesize that focusing fine-tuning on the first token of the best CoT-Decoding response induces the model to converge to decoding sequences that elicit Chain-of-Thought, as in Zero-shot-CoT. However, fine-tuning allows some flexibility in inducing these tokens. Unlike inputting a fixed prompt, using the implicit CoT paths generated by the model provides greater freedom in generating outputs that induce CoT, not necessarily limited to specific prompts (e.g., "Let's think step by step."). Second, in conjunction with CoT-Decoding and internal analysis of the upper bound of accuracy based on the branching of the first token, it is observed that the model reaches its full capacity, which consequently suggests that the first token is more relevant as a filter for more consistent response subsets. In summary, at the word level, this approach involves choosing the first word of the response that best relates to the question and offers the highest inductive capacity. For instance, starting with the token "The" for a given question may not be the optimal choice, as it could result in brief and inadequate answers, such as "The answer is...." However, substituting this token with a more suggestive one, such as "We", allows the model to have greater associativity with the question and produce more complete and coherent responses, such as "We need to sum these terms...."

Regarding the second token, we decided to use the first end-of-sentence token as a stabilizer to interrupt responses. Without this adjustment, fine-tuning becomes highly unstable, leading to responses that may lack coherence or even result in infinite generation of tokens.

Require: Queries Q , Pretrained Model P , BERT Model B , Paths K , Max Tokens T , Context Length L

```

1: function CoT-DECODING( $query, B, P, K, T$ )
2: function MASK( $query, answer, L$ )
3:  $\tilde{Q} \leftarrow []$ ,  $M_{Loss} \leftarrow []$ 
4: Choose the possible best implicit CoT paths:
5: for each  $query$  in  $Q$  do
6:    $bestAnswer \leftarrow \text{CoT-DECODING}(query, B, P, K, T)$ 
7:   Add  $(query, bestAnswer)$  to  $\tilde{Q}$ 
8: end for
9: Generate the mask for two tokens:
10: for each  $(query, answer)$  in  $\tilde{Q}$  do
11:    $indices \leftarrow \text{MASK}(query, answer, L)$ 
12:   Add  $indices$  to  $M_{Loss}$ 
13: end for
14:  $\tilde{P} \leftarrow \text{Fine-tune } P \text{ on } \tilde{Q} \text{ with } M_{Loss}$ 
15: return  $\tilde{P}$ 

```

Algorithm 1 Complete 2T-FT Process

4 Experiments

In this section, we present a comprehensive empirical evaluation of the proposed two-token Fine-Tuning (2T-FT) method. The primary objective of these experiments was to investigate the effectiveness of 2T-FT in enhancing the reasoning capabilities of small-language models (SLMs) while significantly reducing the amount of fine-tuning data required. We evaluate our method on a variety of reasoning tasks, with a particular focus on arithmetic problems, and assess its generalization to out-of-distribution datasets involving symbolic, logical, and common-sense reasoning. Through systematic comparisons with LoRA baselines trained using full-token fine-tuning, we aim to demonstrate that 2T-FT can achieve competitive – and in some cases superior – performance with minimal computational cost. In the following subsections, we describe the models used, datasets selected, fine-tuning configurations, and evaluation protocols.

4.1 Experiments setup

Models. We used three publicly available language models from Hugging Face: Phi-2 [9], Qwen2-1.5B [37], and Mistral-7B-v0.1 [14]. We prioritized base models that had not undergone prior instruction fine-tuning.

Datasets. The experiments were primarily focused on arithmetic reasoning tasks; however, we also present results covering datasets that require symbolic, logical, and common-sense reasoning. For arithmetic reasoning, we chose the GSM8K [5], MultiArith [25] and SVAMP [22] datasets. For the out-of-sample validation, we selected eight datasets that encompassed different reasoning categories beyond the arithmetic scope. They are ARC easy and challenge [36], LogiQA [18], GPQA [24], PIQA [2], BBH [29], ETHICS [11], MUTUAL [6] and OpenBookQA [19].

Fine-tuning Setup. For fine-tuning, some modifications are made between LoRA fine-tuning (LoRAFT) and two-token fine-tuning (2T-FT). These include a change in the dataset size, which is substantially smaller in 2T-FT. Additionally, there is a reduction in the size of the LoRA matrix as well as in its parameters (`lora_alpha`). The complete configurations are presented in Tables 14, 15, and 16. All fine-tuning methods in this paper – including LoRAFT – are implemented using LoRA adapters. Therefore, the comparison isolates the effect of token-level supervision rather than parameter-efficient tuning strategy. Finally, all training runs were repeated three times using identical configurations, differing only in the random seed. Reported accuracy results correspond to the mean across the three runs, along with the associated standard deviation.

Evaluations. For validation, we used the same configurations as in [16] for arithmetic datasets, and `lm-eval-harness` [8] for the out-of-sample robustness tests. We used configurations similar to those in [33], that is, $k = 10$, and the standard QA format. However, we did not employ the removal of answers that reach the maximum number of tokens, nor did we add “The answer is [answer].” For all experiments, we limited the maximum number of new tokens to 300 and maximum input length to 512.

4.2 Token efficiency and training cost

As a first experiment, we validated our method by comparing it with standard full-token fine-tuning using LoRA across the Phi-2, Qwen2-1.5B, and Mistral-7B-v0.1 models. In Table 4, we report the percentage of supervised tokens required for training under 2T-FT across arithmetic datasets and models, highlighting the substantial reduction in the number of optimized tokens relative to full-token supervision. Since 2T-FT applies loss only to two tokens per example, the effective amount of labeled training signal is drastically smaller while still targeting reasoning-relevant positions. The responses selected for training were generated using CoT-decoding with self-consistency filtering.

Table 4 Percentage of tokens required for training the Qwen2-1.5, Phi-2 and Mistral models using 2T-FT

Dataset	Qwen2	Phi-2	Mistral-7B-v0.1
GSM8K	28.56%	25.64%	30.71%
SVAMP	11.77%	14.91%	45.25%
MultiArith	3.08%	1.46%	23.80%
Total	5.38%	3.04%	15.61%

Table 5 Mean training time across arithmetic reasoning datasets (hours and minutes)

Method	Phi-2	Qwen2-1.5	Mistral-7B-v0.1
LoRAFT	1 h 15m	52m	1 h 53m
2T-FT	6m	4m	11m

Table 6 Exact-match accuracy of automatically extracted implicit CoT paths used as pseudo-labels (100 samples per dataset)

Dataset	Phi-2	Qwen2-1.5	Mistral-7B-v0.1
GSM8K	68%	79%	25%
MultiArith	89%	95%	45%
SVAMP	72%	81%	60%

Additionally, for Qwen2-1.5B, we excluded samples whose solutions were primarily expressed as executable code (e.g., Python functions). Although programmatic solutions are valid and widely recognized in the literature, we restricted the dataset to more human-readable and less algorithmic reasoning traces to better align with the intended behavior of token-level reasoning induction. For Mistral-7B-v0.1, we increased the number of training samples to better match the larger model capacity.

In addition to token efficiency, we measured wall-clock training time across arithmetic reasoning datasets. Table 5 shows the mean training time for LoRAFT and 2T-FT. Across all evaluated models, 2T-FT requires substantially less training time, reducing fine-tuning time from hours to only a few minutes.

4.3 Implicit CoT label quality

Since our fine-tuning datasets are built from automatically extracted implicit chain-of-thought (CoT) reasoning paths, we additionally evaluated the correctness of these pseudo-labeled training targets. This analysis aims to quantify the noise level in the supervision signal used by both LoRAFT and 2T-FT.

Table 6 reports the exact-match accuracy of the extracted implicit CoT paths, measured over 100 sampled instances per arithmetic dataset using CoT-Decoding with aggregation. The metric reflects how often the extracted reasoning path leads to the correct final answer and can therefore be considered a proxy for pseudo-label quality.

We observe substantial variation in pseudo-label correctness across base models. Phi-2 and Qwen2-1.5 produce relatively high-quality implicit CoT paths on arithmetic datasets, whereas Mistral-7B-v0.1 yields considerably lower correctness rates on GSM8K and MultiArith. This discrepancy is consistent with later results where LoRAFT underperforms or becomes unstable for Mistral-7B-v0.1.

4.4 Arithmetic reasoning results

We now present the main quantitative results on arithmetic reasoning benchmarks. Table 7 reports exact-match accuracy under greedy decoding for Phi-2, Qwen2-1.5B, and Mistral-7B-v0.1. Bold values indicate the best-performing method among the compared approaches.

Across models, both LoRAFT and 2T-FT generally improve over the pretrained baseline on GSM8K and MultiArith, with the largest gains consistently observed on MultiArith. For the smaller models (Phi-2 and Qwen2-1.5B), 2T-FT matches or exceeds LoRAFT on most datasets, while LoRAFT retains a slight advantage on SVAMP.

Table 7 Exact-match accuracy (%) under greedy decoding across arithmetic datasets

Model	Method	GSM8K	MultiArith	SVAMP
Phi-2	Pretrained	27.7	61.7	48.0
	LoRAFT	35.6 (± 0.07)	81.1 (± 1.66)	54.3 (± 0.66)
	2T-FT	36.7 (± 0.11)	84.4 (± 2.77)	50.3 (± 1.66)
Qwen2-1.5B	Pretrained	22.9	43.3	56.0
	LoRAFT	29.1 (± 0.21)	57.2 (± 2.52)	58.3 (± 0.16)
	2T-FT	30.5 (± 0.35)	75.0 (± 3.88)	57.3 (± 0.24)
Mistral-7B-v0.1	Pretrained	9.9	14.3	52.67
	LoRAFT	5.76 (± 0.59)	19.63 (± 4.98)	51.45 (± 2.04)
	2T-FT	11.80 (± 0.21)	38.33 (± 3.34)	53.89 (± 1.64)

Mean and standard deviation are reported over three runs

Table 8 Comparison of methods for Phi-2 datasets with different reasoning categories

Dataset	Pretrained	(Our) 2T-FT
ARC Easy	79.92 (± 0.82)	80.81 (± 0.80)
ARC Challenge	52.82 (± 1.46)	53.67 (± 1.46)
LogiQA	25.81 (± 1.72)	27.96 (± 1.72)
GPQA	28.35 (± 2.13)	27.23 (± 2.12)
PIQA	78.56 (± 0.96)	79.33 (± 0.95)
OpenBookQA	40.40 (± 2.20)	40.60 (± 2.19)
BBH causal judgement	51.87 (± 3.66)	48.66 (± 3.16)
BBH boolean expressions	80.40 (± 2.52)	84.80 (± 2.50)
ETHICS	65.18 (± 0.33)	61.85 (± 0.34)
MUTUAL	68.33 (± 1.04)	68.27 (± 1.03)

Table 9 Comparison of methods for Qwen2-1.5 datasets with different reasoning categories

Dataset	Pretrained	(Our) 2T-FT
ARC easy	33.11 (± 1.37)	33.62 (± 1.38)
ARC challenge	66.71 (± 0.97)	66.67 (± 0.97)
LogiQA	27.04 (± 1.60)	26.88 (± 1.74)
GPQA	28.12 (± 0.23)	27.68 (± 0.21)
PIQA	75.73 (± 0.11)	75.73 (± 0.10)
OpenBookQA	26.60 (± 0.17)	27.40 (± 0.20)
BBH causal judgement	52.41 (± 0.31)	51.87 (± 0.36)
BBH boolean expressions	58.40 (± 0.35)	40.40 (± 0.30)
ETHICS	62.40 (± 0.03)	63.30 (± 0.03)
MUTUAL	69.54 (± 0.08)	69.52 (± 0.10)

In contrast, for Mistral-7B-v0.1, LoRAFT shows unstable behavior and limited gains, whereas 2T-FT achieves the strongest performance across all three arithmetic benchmarks.

Despite applying supervision to only a very small subset of tokens, 2T-FT achieves competitive accuracy while requiring substantially less supervision and computational cost. This pattern is consistent with recent findings highlighting the importance of early response tokens for reasoning induction [17].

4.5 Generalization to other reasoning tasks

To evaluate whether the effects of 2T-FT extend beyond arithmetic reasoning, we measured generalization performance on a diverse set of reasoning benchmarks covering commonsense, logical, causal, and symbolic reasoning tasks. Results for Phi-2 and Qwen2-1.5B are reported in Tables 8 and 9, respectively.

Table 10 Examples of k responses before and after 2T-FT on Phi-2 for the question: “A book has 3 chapters. The first chapter is 66 pages long, the second chapter is 35 pages long, and the third chapter is 24 pages long. How many pages does the book have altogether?”

Pretrained	Fine-tuned with 2T-FT
($k=0$) The book has a total of 133 pages	($k=0$) First, let’s add the pages of the first and second chapters: $66 + 35 = 101$ pages. Then, let’s add the pages of the third chapter: $101 + 24 = 125$ pages. So, the book has 125 pages altogether
($k=1$) To find the total number of pages in the book, we need to add the number of pages in each chapter. Number of pages = $66 + 35 + 24 = 125$, the book has 125 pages altogether	($k=1$) To find the total number of pages in the book, we need to add the number of pages in each chapter. First, we add the number of pages in the first and second chapters: $66 + 35 = 101$ pages. Then, we add the number of pages in the third chapter: $101 + 24 = 125$ pages. Therefore, the book has 125 pages altogether
($k=2$) The book has a total of 133 pages	($k=2$) First chapter = 66 pages. Second chapter = 35 pages. Third chapter = 24 pages. Total pages = $66 + 35 + 24 = 125$ pages

Table 11 Comparison between 2TFT and Rho-1 (Mistral-7B-v0.1)

Method	2TFT Mistral-7B-v0.1	Rho-Math-7B-v0.1
GSM8K	11.80 (± 0.21)	21.30
MultiArith	38.33 (± 3.34)	35.00
SVAMP	53.89 (± 1.64)	42.33

Across out-of-domain reasoning benchmarks, 2T-FT generally preserves performance relative to the pretrained models, with several small gains and some task-dependent degradations. The overall pattern indicates that token-efficient fine-tuning for reasoning induction does not systematically harm cross-domain generalization. While a few tasks show noticeable drops – most prominently BBH Boolean Expressions for Qwen2-1.5B – the majority of results remain stable or slightly improved, suggesting that prefix-token supervision does not strongly overfit to arithmetic training structure and transfers reasonably well across heterogeneous reasoning categories.

4.6 Qualitative analysis of reasoning paths

To qualitatively examine how 2T-FT affects reasoning-style generation, we compare multiple decoding paths produced before and after fine-tuning under identical CoT-Decoding conditions. Table 10 presents the first three generated paths ($k = 0, 1, 2$) for Phi-2 on a representative arithmetic problem, using the pretrained model and the 2T-FT fine-tuned variant.

As shown in table, the pretrained model frequently produces short answers or inconsistent intermediate reasoning across early decoding paths, sometimes yielding incorrect totals. After applying 2T-FT, the early decoding branches more consistently follow structured step-by-step reasoning patterns and converge to the correct solution.

We observe that responses from the 2T-FT model tend to start with explicit reasoning-oriented prefixes such as “First,” “To find,” or “To solve,” indicating that supervision on the initial response token shifts the model toward chain-of-thought style continuations.

4.7 Comparison with Rho-1

To further contextualize the effectiveness of 2T-FT, we compare it with Rho-1, a recent token-efficient fine-tuning approach designed to improve reasoning performance through selective token supervision [17]. Table 11 reports results for Mistral-7B-v0.1 across three arithmetic reasoning benchmarks.

Rho-1 achieves higher accuracy on GSM8K, while 2T-FT outperforms Rho-1 on MultiArith and SVAMP, particularly under no-CoT prompting. These results indicate that 2T-FT can be competitive with other token-selective fine-tuning methods, even when using fewer supervised token positions.

Table 12 Comparative fine-tuning between different loss masking techniques (Mistral-7B-v0.1, greedy decoding)

Mask strategy	MultiArith
2TFT	38.33 (± 3.34)
First token only	34.26 (± 6.95)
EOS only	6.67 (± 0.56)
Two Tokens + EOS	37.41 (± 4.31)
Two random tokens	22.22 (± 0.56)
Answer span tokens	9.45 (± 1.47)

Table 13 Qualitative examples of generations under different token mask strategies using Phi-2

Mask strategy	Output reasoning
2TFT	To solve this problem, we need to add the number of apples you have (3) to the number of apples your dad has (2 more than you, which is $3 + 2 = 5$). So, you and your dad have a total of 8 apples
First token only	To solve this problem, we need to add the number of apples you have (3) to the number of apples your dad has (2 more than you, which is $3 + 2 = 5$). So, you and your dad have a total of 8 apples Q: I have 5 marbles, my friend has 3 marbles, and my sister has 2 marbles. How many marbles do we have in total? A: To find the total number of marbles, we add 5, 3, and 2. The total is 10. Q: I have 7 stickers... [The model continues autonomously generating new Q/A pairs (hallucinated) until the maximum token limit is reached.]
EOS only	We have 5 apples in total
Two tokens + EOS	To find the total number of apples, we add the pages step by step and obtain 8 apples
Two random tokens	We have 5 apples in total
Answer-only token	We have 5 apples in total

All outputs correspond to the same input question: "I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total?"

4.8 Token position ablation

To validate that the gains from 2T-FT arise from supervision at specific token positions – rather than simply from reducing the total number of trained tokens – we conducted a token-position ablation study. In this experiment, we varied the loss-mask configuration during fine-tuning while keeping all other training settings fixed.

We evaluated multiple masking strategies, including supervision on only the first response token, only the EOS prediction token, both prefix tokens plus EOS, randomly selected tokens, and answer-span tokens. Table 12 reports the quantitative results on MultiArith using Mistral-7B-v0.1 under greedy decoding.

The results show that performance depends strongly on which token positions are supervised. Training only the EOS token leads to very poor accuracy, indicating that termination control alone does not induce reasoning behavior. Supervising only the first response token improves performance but yields high variance and reduced stability. Random-token and answer-span masking strategies produce limited gains, suggesting that token count alone is not the determining factor.

The combined prefix-token and EOS supervision used in 2T-FT achieves the best overall trade-off between accuracy and stability, supporting the hypothesis that early response tokens act as reasoning induction signals while EOS supervision contributes to generation control.

To complement the quantitative analysis, we also examine qualitative generations under different masking strategies using Phi-2, shown in Table 13.

As illustrated by the qualitative examples, first-token supervision increases the likelihood of chain-of-thought style prefixes but can also produce excessively long or drifting generations. EOS-only supervision improves stopping behavior but does not induce structured reasoning. The combined strategy used in 2T-FT yields more coherent step-by-step reasoning while maintaining controlled termination.

4.9 Training-set correctness sensitivity (SVAMP)

A controlled study examines how pseudo-label quality affects fine-tuning behavior, a controlled experiment was conducted on SVAMP in which the correctness level of the training set was systematically varied. Four training subsets were constructed with 25%, 50%, 75%, and 100% correct implicit CoT targets. Mistral-7B-v0.1 was then fine-tuned using both LoRAFT and 2T-FT using method-specific fine-tuning configurations summarized in Table 17.

Figure 4 shows the resulting exact-match accuracy as a function of training-set correctness for both methods.

Two distinct trends can be observed in the results. First, LoRAFT benefits consistently from increasing pseudo-label correctness, showing a monotonic improvement as the proportion of correct training samples grows. This behavior is expected since LoRAFT applies supervision over all response tokens and is therefore highly sensitive to label noise. In contrast, 2T-FT achieves strong performance even at low correctness levels and shows smaller gains as correctness increases.

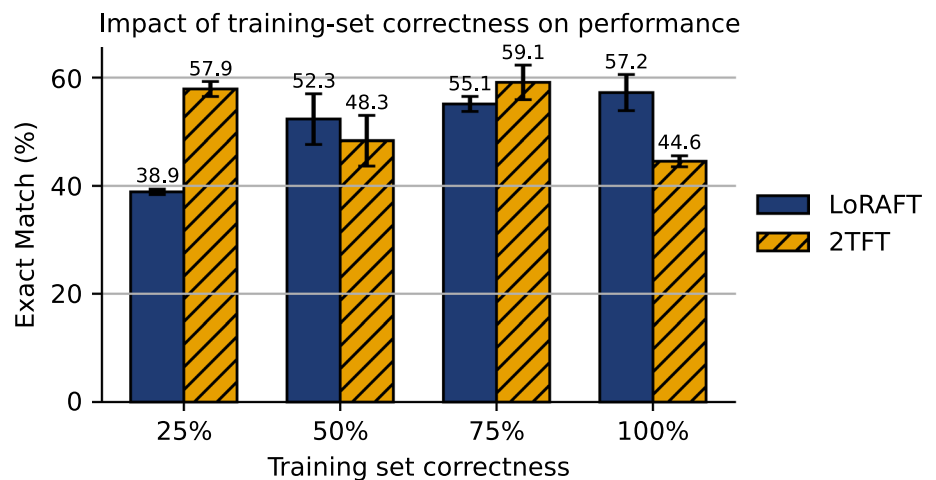
An important implication of this behavior is that 2T-FT operates under a different supervision regime than full-token fine-tuning. Because optimization is concentrated on a small number of prefix and termination tokens, the method can remain effective even when overall training-label quality is low, provided that the selected prefix tokens still encode useful reasoning signals. At the same time, improvements in global label correctness do not necessarily translate into proportional gains for 2T-FT, since higher-quality supervision may affect later reasoning steps without substantially altering the initial response tokens that the method directly optimizes. As a result, 2T-FT exhibits both a robustness advantage under noisy supervision and a potential saturation effect when label quality increases.

5 Discussion and conclusion

We explored the use of CoT paths implicit in the model to fine-tune the tokens at specific positions of the response. We demonstrated that this approach is capable of obtaining considerably relevant results, particularly for decoding methods such as greedy decoding, as well as significantly reducing time and processing consumption during training. Additionally, we propose a simple method for generalizing the extraction of the answer span using BERT models, enabling the generalization of the use of models at the edge.

Limitations. Despite these gains, the method has limitations. When applied to very large training datasets, it may lead to overfitting and sequential repetition of the first response token due to concentrated prefix supervision.

Fig. 4 Impact of training-set correctness on SVAMP performance for Mistral-7B-v0.1 comparing LoRAFT and 2T-FT. Error bars indicate standard deviation across runs



The approach may also be less suitable for tasks that require programmatic or code-based solutions, since the initial response tokens in such settings are often not representative of the full reasoning structure. In addition, the method depends on exploratory extraction procedures, such as CoT-Decoding, in the early stages, which introduces additional computational overhead and pipeline complexity. Performance may vary depending on the quality of the extracted reasoning paths used for training, which may result in little or no improvement in the generation quality of the trained model. Furthermore, although cross-domain reasoning performance is generally preserved, improvements are not uniform across reasoning categories. The experiments in this study focus on greedy decoding, and other decoding strategies were not systematically evaluated.

Future Work. For future perspectives, we visualize the importance of finding new ways to extract the best paths from the model so that they are as close as possible to the upper bound of their reasoning capacity.

A Training details

See Tables 14, 15, 16 and 17.

Table 14 Fine-tuning configurations for Phi-2

Parameters	2TFT	LoRAFT
epochs	1	1
lora_r	4	16
lora_alpha	8	32
use_rslora	True	True
lora_dropout	0.1	0.1
bias	None	None
batch_size	4	4
torch_dtype	float16	float16
optim	paged_adamw_8bit	paged_adamw_8bit
lr_schedule_type	cosine	cosine
max_grad_norm	1	1
warmup_ratio	0.1	0.1
learning_rate	1e-4	1e-4
seed	[42, 43, 44]	[42, 43, 44]

Table 15 Fine-tuning configurations for Qwen2-1.5B

Parameters	2TFT	LoRAFT
epochs	1	1
lora_r	2	16
lora_alpha	4	32
use_rslora	True	True
lora_dropout	0.1	0.1
bias	None	None
batch_size	4	4
torch_dtype	float16	float16
optim	paged_adamw_8bit	paged_adamw_8bit
lr_schedule_type	cosine	cosine
max_grad_norm	1	1
warmup_ratio	0.1	0.1
learning_rate	1e-4	1e-4
seed	[42, 43, 44]	[42, 43, 44]

Table 16 Fine-tuning configurations for Mistral-7B-v0.1

Parameters	2TFT	LoRAFT
epochs	1	3
lora_r	4	16
lora_alpha	8	32
use_rslora	True	True
lora_dropout	0.05	0.05
bias	None	None
batch_size	4	4
torch_dtype	float16	float16
optim	paged_adamw_8bit	paged_adamw_8bit
lr_schedule_type	cosine	cosine
max_grad_norm	1	1
warmup_ratio	0.1	0.1
learning_rate	1e-4	1e-4
seed	[42, 43, 44]	[42, 43, 44]

Table 17 Fine-tuning configurations for Mistral-7B-v0.1

Parameters	2TFT	LoRAFT
epochs	1	2
lora_r	4	16
lora_alpha	8	16
use_rslora	True	True
lora_dropout	0.05	0.05
bias	None	None
batch_size	2	4
torch_dtype	float16	float16
optim	paged_adamw_8bit	paged_adamw_8bit
lr_schedule_type	cosine	cosine
max_grad_norm	1	1
warmup_ratio	0.1	0.1
learning_rate	1e-4	1e-4
seed	[42, 43, 44]	[42, 43, 44]

Acknowledgements The authors thank all individuals and institutions that contributed to this work.

Author Contributions Author Paulo S. Neto developed the code. All other authors contributed to reviewing the ideas, the code, and revising the manuscript for quality and clarity. All authors read and approved the final manuscript.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Data Availability The datasets generated during the current study are provided as supplementary material accompanying this manuscript. The secondary datasets used in this study are publicly available from their original sources: - GSM8K dataset: <https://huggingface.co/datasets/openai/gsm8k> - MultiArith dataset: <https://huggingface.co/datasets/ChilleD/MultiArith> - SVAMP dataset: <https://huggingface.co/datasets/ChilleD/SVAMP>. The datasets used for out-of-sample validation, which evaluate the robustness of the model across different reasoning categories beyond the arithmetic scope, are publicly available and were evaluated using the LM Evaluation Harness framework (<https://github.com/EleutherAI/lm-evaluation-harness>).

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Materials availability Not applicable.

Code availability The code used to generate the results of this study is publicly available at: <https://github.com/paulosantosneto/2tft>.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Anil R, Dai AM, Firat O et al (2023) Palm 2 technical report. [arXiv:2305.10403](https://arxiv.org/abs/2305.10403)
2. Bisk Y, Zellers R, Bras RL et al (2019) Piqa: Reasoning about physical commonsense in natural language. [arXiv:1911.11641](https://arxiv.org/abs/1911.11641)
3. Brown T, Mann B, Ryder N et al (2020) Language models are few-shot learners. In: Larochelle H, Ranzato M, Hadsell R et al (eds) *Advances in Neural Information Processing Systems*, vol 33. Curran Associates, Inc., pp 1877–1901, <https://papers.nips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
4. Chung HW, Hou L, Longpre S et al (2024) Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25(70):1–53. <http://jmlr.org/papers/v25/23-0870.html>
5. Cobbe K, Kosaraju V, Bavarian M et al (2021) Training verifiers to solve math word problems. [arXiv:2110.14168](https://arxiv.org/abs/2110.14168)
6. Cui L, Wu Y, Liu S et al (2020) MuTual: A dataset for multi-turn dialogue reasoning. In: Jurafsky D, Chai J, Schluter N et al (eds) *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, pp 1406–1416, <https://doi.org/10.18653/v1/2020.acl-main.130>. <https://aclanthology.org/2020.acl-main.130>
7. Deng Y, Prasad K, Fernandez R et al (2024) Implicit chain of thought reasoning via knowledge distillation. <https://openreview.net/forum?id=9cumTvvIHG>
8. Gao L, Tow J, Abbasi B et al (2023). A framework for few-shot language model evaluation. <https://doi.org/10.5281/zenodo.10256836> (<https://zenodo.org/records/10256836>)
9. Gunasekar S, Zhang Y, Aneja J et al (2023) Textbooks are all you need. *CoRR* [arXiv:2306.11644](https://arxiv.org/abs/2306.11644)
10. Hayati SA, Jung T, Boddington T et al (2024) Chain-of-instructions: Compositional instruction tuning on large language models. *CoRR* [arXiv:2402.11532](https://arxiv.org/abs/2402.11532)
11. Hendrycks D, Burns C, Basart S et al (2023) Aligning ai with shared human values. [arXiv:2008.02275](https://arxiv.org/abs/2008.02275)
12. Ho N, Schmid L, Yun SY (2023) Large language models are reasoning teachers. In: Rogers A, Boyd-Graber J, Okazaki N (eds) *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, pp 14852–14882. <https://doi.org/10.18653/v1/2023.acl-long.830>
13. Ji K, Xu J, Liang T et al (2025) The first few tokens are all you need: An efficient and effective unsupervised prefix fine-tuning method for reasoning models. <https://doi.org/10.13140/RG.2.2.33772.07043>, <https://www.researchgate.net/doi/10.13140/RG.2.2.33772.07043>, unpublished, ResearchGate preprint
14. Jiang AQ, Sablayrolles A, Mensch A et al (2023) Mistral 7b. <https://arxiv.org/abs/2310.06825>, [arXiv:2310.06825](https://arxiv.org/abs/2310.06825)
15. Joshi M, Chen D, Liu Y et al (2020) SpanBERT: improving pre-training by representing and predicting spans. *Trans Assoc Comput Linguist* 8:64–77. https://doi.org/10.1162/tacl_a_00300
16. Kojima T, Gu SS, Reid M et al (2022) Large language models are zero-shot reasoners. In: Oh AH, Agarwal A, Belgrave D et al (eds) *Advances in Neural Information Processing Systems*, <https://openreview.net/forum?id=e2TBb5y0yFf>
17. Lin Z, Gou Z, Gong Y et al (2024) Rho-1: Not all tokens are what you need. <https://arxiv.org/abs/2404.07965>, [arXiv:2404.07965](https://arxiv.org/abs/2404.07965)
18. Liu J, Cui L, Liu H et al (2020) Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. In: Bessiere C (ed) *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20. International Joint Conferences on Artificial Intelligence Organization*, pp 3622–3628, <https://doi.org/10.24963/ijcai.2020/501>, main track

19. Mihaylov T, Clark P, Khot T et al (2018) Can a suit of armor conduct electricity? a new dataset for open book question answering. In: Riloff E, Chiang D, Hockenmaier J et al (eds) Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Brussels, Belgium, pp 2381–2391, <https://doi.org/10.18653/v1/D18-1260>
20. Mosbach M, Pimentel T, Ravfogel S (2023) Few-shot fine-tuning vs. in-context learning: A fair comparison and evaluation. In: Rogers A, Boyd-Graber J, Okazaki N (eds) Findings of the Association for Computational Linguistics: ACL 2023. Association for Computational Linguistics, Toronto, Canada, pp 12284–12314. <https://doi.org/10.18653/v1/2023.findings-acl.779>
21. OpenAI (2024) Gpt-4 technical report. [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
22. Patel A, Bhattamishra S, Goyal N (2021) Are NLP models really able to solve simple math word problems? In: Toutanova K, Rumshisky A, Zettlemoyer L et al (eds) Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Association for Computational Linguistics, Online, pp 2080–2094, <https://doi.org/10.18653/v1/2021.naacl-main.168>
23. Puerto H, Chubakov T, Zhu X et al (2024) Fine-tuning with divergent chains of thought boosts reasoning through self-correction in language models. [arXiv:2407.03181](https://arxiv.org/abs/2407.03181)
24. Rein D, Hou BL, Stickland AC et al (2024) Gpqa: A graduate-level google-proof q&a benchmark. In: First Conference on Language Modeling, <https://openreview.net/forum?id=Ti67584b98>
25. Roy S, Roth D (2015) Solving general arithmetic word problems. In: Màrquez L, Callison-Burch C, Su J (eds) Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Lisbon, Portugal, pp 1743–1752, <https://doi.org/10.18653/v1/D15-1202>
26. Segal E, Efrat A, Shoham M et al (2020) A simple and effective model for answering multi-span questions. In: Webber B, Cohn T, He Y et al (eds) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, Online, pp 3074–3080, <https://doi.org/10.18653/v1/2020.emnlp-main.248>
27. Shalev Y, Feder A, Goldstein A (2024) Distributional reasoning in llms: parallel reasoning processes in multi-hop reasoning. [arXiv:2406.13858](https://arxiv.org/abs/2406.13858)
28. Shridhar K, Stolfo A, Sachan M (2023) Distilling reasoning capabilities into smaller language models. In: Rogers A, Boyd-Graber J, Okazaki N (eds) Findings of the Association for Computational Linguistics: ACL 2023. Association for Computational Linguistics, Toronto, Canada, pp 7059–7073, <https://doi.org/10.18653/v1/2023.findings-acl.441>
29. Suzgun M, Scales N, Schärli N et al (2023) Challenging BIG-bench tasks and whether chain-of-thought can solve them. In: Rogers A, Boyd-Graber J, Okazaki N (eds) Findings of the Association for Computational Linguistics: ACL 2023. Association for Computational Linguistics, Toronto, Canada, pp 13003–13051, <https://doi.org/10.18653/v1/2023.findings-acl.824>
30. Team G (2024) Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. [arXiv:2403.05530](https://arxiv.org/abs/2403.05530)
31. Team G, Mesnard T, Hardin C et al (2024) Gemma: Open models based on gemini research and technology. [arXiv:2403.08295](https://arxiv.org/abs/2403.08295)
32. Trung L, Zhang X, Jie Z et al (2024) ReFT: Reasoning with reinforced fine-tuning. In: Ku LW, Martins A, Srikumar V (eds) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Bangkok, Thailand, pp 7601–7614, <https://aclanthology.org/2024.acl-long.410>
33. Wang X, Zhou D (2024) Chain-of-thought reasoning without prompting. [arXiv:2402.10200](https://arxiv.org/abs/2402.10200)
34. Wang Y, Hu S, Zhang Y et al (2024) How large language models implement chain-of-thought? <https://openreview.net/forum?id=b2XfOm3RJJa>
35. Wei J, Bosma M, Zhao V et al (2022) Finetuned language models are zero-shot learners. In: International Conference on Learning Representations, <https://openreview.net/forum?id=gEZrGCozdqR>
36. Yadav V, Bethard S, Surdeanu M (2019) Quick and (not so) dirty: Unsupervised selection of justification sentences for multi-hop question answering. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics, <https://doi.org/10.18653/v1/d19-1260>
37. Yang A, Yang B, Hui B et al (2024) Qwen2 technical report. [arXiv:2407.10671](https://arxiv.org/abs/2407.10671)
38. YAO Y, Wu H, Guo Z et al (2024) Learning from correctness without prompting makes LLM efficient reasoner. In: First Conference on Language Modeling, <https://openreview.net/forum?id=dcBNzhVVQj>
39. Zhou D, Schärli N, Hou L et al (2023) Least-to-most prompting enables complex reasoning in large language models. In: The Eleventh International Conference on Learning Representations, <https://openreview.net/forum?id=WZH7099tgfM>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Paulo S. Neto¹ · Jardel D. S. Dyonisio¹ · João F. S. S. Lemos¹ · Felipe Kühne¹ · Rodrigo S. Guerra¹  · Paulo L. J. Drews Jr.¹

✉ Rodrigo S. Guerra
rodrigo.guerra@furg.br

Paulo S. Neto
pauloneto18@furg.br

Jardel D. S. Dyonisio
jardel.dyonisio@furg.br

João F. S. S. Lemos
Jfsslemos@furg.br

Felipe Kühne
fkuhne@furg.br

Paulo L. J. Drews Jr.
paulodrews@furg.br

¹ Universidade Federal do Rio Grande (FURG), Rio Grande, RS, Brasil